Deep Adaptive Design of Experiments for Dynamical Systems

Arno Strouwen PumasAI · KU Leuven

Sebastian Micluța-Câmpeanu JuliaHub · University of Bucharest

JuliaCon 2026

Notation and goal

- θ: the unknown parameters.
- u: the design.
- y: the observations.

State of belief *before* the experiment:

$$p(\theta) \qquad \text{the } \textbf{prior}$$

After the experiment, Bayes' rule updates it:

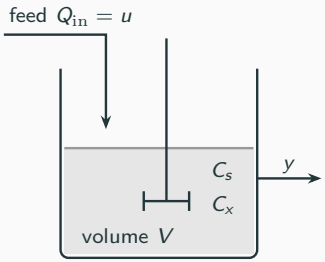
$$p(\theta \mid y, u) = \frac{p(y \mid \theta, u)\, p(\theta)}{\displaystyle\int p(y \mid \theta', u)\, p(\theta')\, d\theta'} \qquad \text{the } \textbf{posterior}$$

Bayesian experimental design: choose u to make the posterior as sharp as possible.

Concretely: a fed-batch bioreactor

$$\dot{C}_s = -\frac{\mu(C_s)}{Y_{xs}}\, C_x + \frac{Q_{\text{in}}}{V}\,(C_{s,\text{in}} - C_s),$$
$$\dot{C}_x = \mu(C_s)\, C_x - \frac{Q_{\text{in}}}{V}\, C_x,$$
$$\dot{V} = Q_{\text{in}}, \qquad \mu(C_s) = \frac{\mu_{\max}\, C_s}{K_s + C_s}.$$

Substrate is pumped in, biomass eats it and grows, and the vessel fills up.



- **Design** $u = Q_{\text{in}}$: the feed rate, which we may change once an hour.
- **Observations** $y = C_s + \text{noise}$: the substrate concentration, sampled once an hour.
- **Unknown** $\theta = (\mu_{\max}, K_s)$: the two constants in the growth law μ.

Which feed profile makes the posterior over $\mu_{\max}$ and K_s as sharp as possible?

Scoring a design

How sharp is a posterior? Measure it by its **entropy**, small when sharp and large when vague:

$$H\big[p(\theta \mid y, u)\big] = -\int p(\theta \mid y, u)\, \log p(\theta \mid y, u)\, d\theta.$$

y is not measured yet, so we average over every observation the design could produce:

$$u^\star = \arg\min_u \; \mathbb{E}_{y|u}\big[\, H[p(\theta \mid y, u)]\,\big].$$

Scoring a design means averaging over the data it might produce.

In our examples only part of θ is of interest: split $\theta = (\theta_T, \theta_N)$ into the kinetics θ_T and nuisance θ_N (sensor noise, initial biomass), and score only the marginal posterior of the target,

$$u^\star = \arg\min_u \; \mathbb{E}_{y|u}\big[\, H[p(\theta_T \mid y, u)]\,\big], \qquad p(\theta_T \mid y, u) = \int p(\theta \mid y, u)\, d\theta_N.$$

We will not go into what that does to the estimator here; it is in the paper: *Deep Adaptive Model-Based Design of Experiments*, arXiv:2603.16146.

Why that is expensive

Up to a constant, the entropy criterion can be rewritten as

$$\max_{u} \; \mathbb{E}_{\theta,\, y \mid u}\big[\, \log p(y \mid \theta, u) - \log p(y \mid u) \big].$$

The first term is easy: simulate the model and evaluate its likelihood.

The second term is the **evidence**, an integral over every parameter the prior allows:

$$p(y \mid u) = \int p(y \mid \theta, u)\, p(\theta)\, d\theta.$$

Replace it by a sample average and you get a loop inside a loop:

$$\frac{1}{N} \sum_{n=1}^{N} \Big[\log p\big(y_n \mid \theta^{(n)}, u\big) \; - \; \log \underbrace{\frac{1}{L} \sum_{\ell=1}^{L} p\big(y_n \mid \theta^{(\ell)}, u\big)}_{\text{inner loop}} \Big]$$

Every one of those likelihoods is an ODE solve.

$N \times L$ ODE solves to score *one* candidate design, and the optimizer wants many.

From static schedules to policies

Static: choose the whole input sequence $u_{1:K} = (u_1, \dots, u_K)$ before the experiment.

Adaptive: choose u_k only after seeing the history h_{k-1} of past inputs and measurements. The optimum alternates a decision and an expectation at every step:

$$V = \min_{u_1} \mathbb{E}_{y_1} \min_{u_2} \mathbb{E}_{y_2} \cdots \min_{u_K} \mathbb{E}_{y_K} \big[\, H[p(\theta \mid h_K)] \,\big].$$

The usual workaround: after each measurement, update the posterior and re-optimize only the next input — inference and optimization, between two measurements.

Instead: let a neural network be the rule, $u_k = \pi_\phi(h_{k-1})$, and optimize its weights ϕ by policy gradient. The nesting collapses into one expectation over simulated episodes, and the best policy is as good as V.

Trained once, offline; in the lab, each step is one forward pass.

What one gradient step costs

One decent estimate of the criterion (a contrastive bound, sPCE) takes $L + M + 2$ ODE solves per episode — one rollout, $L = 464$ prior decoys, $M = 466$ nuisance draws — over a batch of $B = 53{,}648$ episodes:

$$\underbrace{932}_{L+M+2} \times \underbrace{53{,}648}_{B} \approx 5 \times 10^7 \text{ trajectories} \qquad \approx 3.5 \times 10^{10} \text{ RK4 steps.}$$

And we need its gradient, not its value — a thousand times, once per optimizer step.

That is a lot of ODE solves: 15.4 s per gradient step, 4.3 hours on one H100.

So the batch has to live on a GPU, the whole objective has to become one compiled program, and the solver has to be differentiated for us.

The stack

Lux.jl	The policy. Parameters and state are explicit arguments, so the loss is a pure function of the weights — what a tracing compiler needs.
RK4, by hand	Fixed step, batched over hypotheses and episodes. `SimpleDiffEq`'s RK4 drops in unchanged.
Reactant.jl	Traces the *entire* objective — rollout, ODE solves, likelihoods, log-sum-exp, attention — into StableHLO MLIR, then compiles it with XLA.
Enzyme	Reverse-mode differentiation of that traced program, at the MLIR level.
Optimisers.jl	Adam, cosine schedule, gradient accumulation, through `Lux.Training.single_train_step!`.

One gradient step becomes one compiled executable: no Julia left in the inner loop.

The stack, honestly

What we got.

- The code stayed the code: ordinary loops and mutation, no `vmap`, no functional rewrite.
- Steady-state runtime is fast: 3.5×10^{10} differentiated RK4 steps per iteration, in 15.4 s.

What it cost.

- Errors come back from the compiler, not your program: MLIR, not a Julia stack trace.
- Compile time is minutes, once per shape — precisely when you want to iterate.
- The bad one: the gradient was silently wrong. The fix existed, in a version that needed Julia 1.12; we were on 1.11. We upgraded — and met a different bug 1.11 never had.

Pin every version, and check the gradient against an unfused reference before trusting a run.

The policy: a small transformer over the history

```
const policy = @compact(
    input_proj  = Dense(2 => 32),
    rms1 = RMSNorm((32,)),  mha = MultiHeadAttention(32; nheads = 4),
    rms2 = RMSNorm((32,)),  ff  = Chain(Dense(32 => 64, gelu), Dense(64 => 32)),
    output_head = Dense(32 => 1; init_bias = (rng, dims...) -> fill(-4.0f0, dims...)),
) do x                                          # x: (2, K, B) -- the (y_k, u_k) pairs so far
    x = input_proj(x)
    x = x .+ reshape(sinusoidal_pe(size(x, 2)), 32, :, 1)     # make time explicit
    attn, _ = mha(rms1(x))
    x = x + attn
    x = x + ff(rms2(x))
    @return ACTION_HI .* sigmoid.(output_head(x[:, end, :]))  # next feed rate in [0, 10]
end
```

The history is a fixed 14-slot buffer, zero where nothing has been measured yet, so the future cannot leak in. The positional encoding is where we deviate from DAD: for a dynamical system, hour 3 and hour 11 are not interchangeable. 14k parameters: next to the ODEs, free.

The objective, as written

```
for step in 1:N_STEPS                                  # roll out under the current policy
    action, st = model(input_buffer, ps, st)
    u = integrate(u, θ_dyn_true, action, DT, N_SUBSTEPS)
    y = observe_noisy(u, θ_obs_true, ε, step)          # y = g(x) + σ·ε, with ε drawn up front
    observations[step, :] .= y                         # mutation is fine: it gets traced away
    input_buffer[1, step, :] .= y
    input_buffer[2, step, :] .= action[1, :]
end

for step in 1:N_STEPS                                  # all L+1 hypotheses at once: u is (3, L+1, B)
    u_denom = integrate(u_denom, θ_dyn_denom, designs[step:step, :], DT, N_SUBSTEPS)
    log_likelihood_step!(ll_denom, observations[step:step, :], u_denom, θ_obs_denom)
end

_, loss, _, train_state = Lux.Training.single_train_step!(   # Enzyme, on the traced program
    AutoEnzyme(), targeted_spce_loss, data, train_state)
```

Hypotheses are one more array axis, no `vmap`; the only compiler-facing line is the last. (Condensed from `src/common.jl`.)

The same inner loop in JAX

Julia + Reactant

```
function integrate(u, θ, d, dt, n_sub)
    @trace mincut=true for _ in 1:n_sub
        u = rk4_step(u, θ, d, dt / n_sub)
    end
    return u
end
```

Python + JAX

```
@jax.checkpoint
def integrate_scan(u, theta, Q_in):
  dt_sub = DT / N_SUBSTEPS
  def body(u, _):
    return rk4_step(u, theta, Q_in, dt_sub), None
  u, _ = lax.scan(body, u, None, length=N_SUBSTEPS)
  return u
```

- Both sides emit StableHLO and hand it to XLA: identical arithmetic, different front end.
- Reverse-mode memory: `jax.checkpoint` by hand, versus `mincut=true` letting Enzyme choose the cut.
- Same trade in the rollout: a `for` loop mutating `input_buffer`, versus a `scan` carrying it and `.at[].set()`.
- Over the hypothesis axis both ended up hand-batched anyway, `vmap` or not.

Reactant versus JAX at a matched workload

Same estimator, same shapes: $L = 255$, $M = 256$, $B = 512$, Float32, 14 steps of 50 RK4 substeps — 262,656 trajectories per timed step, on one GPU.

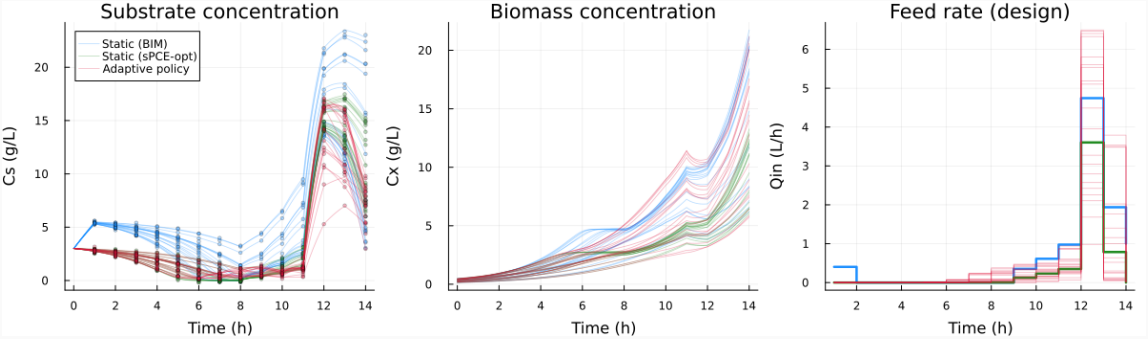
	step	first call
Reactant, hand RK4	—	—
Reactant, SimpleDiffEq	—	—
JAX, `lax.scan`	—	—
JAX, `fori_loop`	—	—

The contract, fixed before running: one deterministic fixture, on the device before the clock starts; time loss and gradient only, no sampling or transfer or optimizer step; synchronize before stopping; five warmups, thirty timed calls, five fresh processes per variant.

Steady-state step time and first-call latency are different numbers, and one matched workload on one GPU is a statement about this program, not about the two languages.

Harness on branch `smc/jax-reactant-benchmark`; numbers pending the matched-fixture rerun.

What the learned policy does

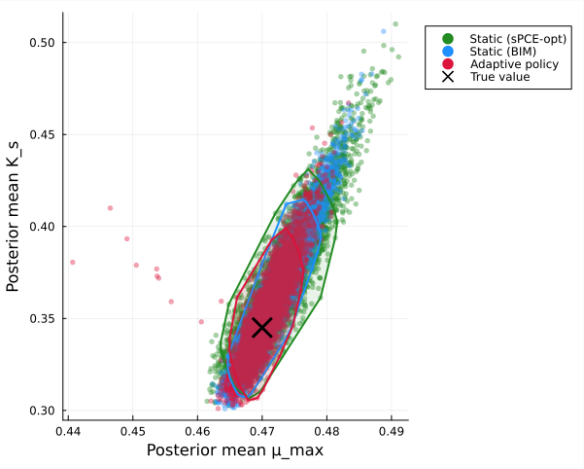


Twenty draws of the true $(\mu_{\max}, K_s)$, the same twenty for each strategy.

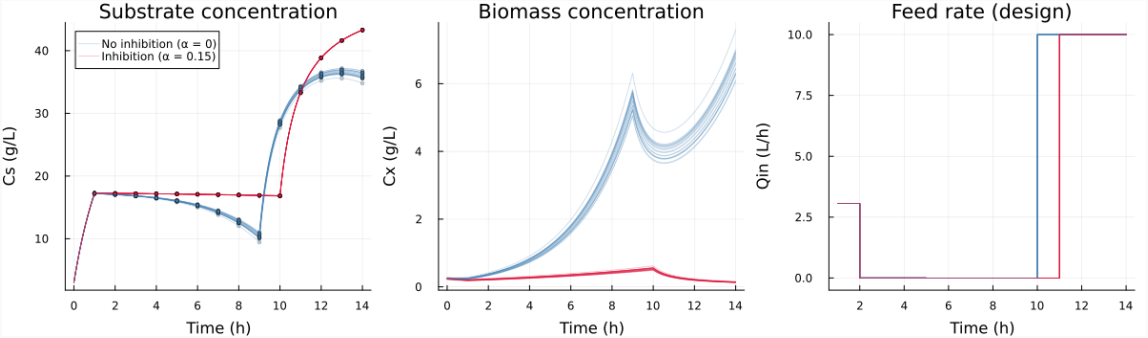
Sharper posteriors

Design	sPCE (nats)	RMSE ($\times 10^3$) $\mu_{\max}$	K_s
Adaptive policy	**3.68**	**3.1**	**24.9**
Static (BIM)	3.45	4.4	34.3
Static (sPCE-opt)	3.34	5.6	41.5

Same number of measurements,
about 30% lower error.



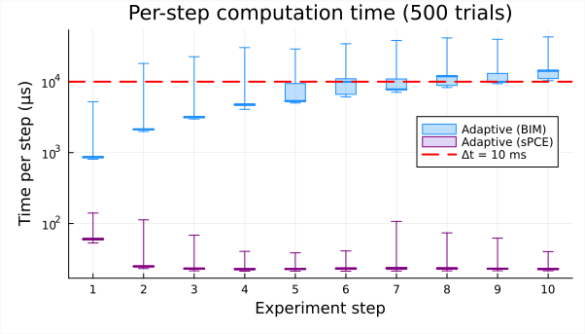
Two runs, two different experiments



Kinetics fixed; two values of the inhibition strength, twenty nuisance draws each.

The design adapts to the true parameters.

Real time: a DC motor on a 10 ms clock



Adaptive (BIM) — re-optimized online. After every measurement: update the posterior, optimize the next voltage. All of it inside the 10 ms window.

Adaptive (sPCE) — the amortized policy. All of that happened offline, before the motor was switched on. Online, one forward pass.

Amortization is what makes adaptive design real-time.

Code and paper

`github.com/arno-papers/CDC2026`

Models, training, evaluation and every figure in this talk.

Paper: `arXiv:2603.16146`

The resources and services used in this work were provided by the VSC (Flemish Supercomputer Center), funded by the Research Foundation – Flanders (FWO) and the Flemish Government.